Unit - 2

Derived Syntactical Constructs in Java

(18 Marks)

Unit-II Derived Syntactical Constructs in Java	2a. Use Constructors for the given programming problem. 2b. Identify scope and lifetime of a variable in the given program code. 2c. Describe the given visibility control with example. 2d. Write the programs by implementing arrays to solve the given problem. 2e. Develop programs using vectors and wrapper classes for the given problem.	2.1 Constructors and methods, types of constructors, nesting of methods, argument passing the 'this' keyword, command line arguments, varargs: variable-length arguments, garbage collection, finalize() method, the object class. 2.2 Visibility Control Public, Private, Protected, default, friendly private Protected access. 2.3 Arrays and Strings: Types of arrays, creating an array, strings, string classes and string buffer, vectors, wrapper classes, enumerated types.

CONSTRUCTOR

1) It is special member function whose task is to initialize the member variables.

2) Constructor name and class name is same.

3) Constructor doesn't having any return type or void

4) Variable are initialize with some value.

5) One class can have more than one constructor

6) Constructor invoked when object is to be initialized

Types: 1) Default Constructor
2) Parameterized Constructor
3) Copy Constructor

NORMAL CONSTRUCTOR- SYNTAX

```
class class_name
{
    variables;
                    // instance members
   constructor_name ()
   {
        //instance member initialization;
    Instance member= value/variable/expression;
   }
}
```

DEFAULT CONSTRUCTOR- EXAMPLE:

```
class Abc
{
    int a;
    float b;                // class instance
    String c;
    Abc()                   // as constructor
    {
     a=5;
     b=12.2;                // member initialization
     c="TYIF";
    }

}
```

```java
class Student
{
    int roll;
    String name;
    double per;

    Student()
    {
        roll=12;
        name="KKWAGH";
        per=88.90;
    }
        void display()
        {
            System.out.println("Roll no: "+roll);
            System.out.println("Name: "+name);
            System.out.println("per: "+per);
        }
}
class College
{
    public static void main(String arga[])
    {
        Student s=new Student();
        s.display();
    }
}
```

PARAMETERIZED CONSTRUCTOR- SYNTAX

```
class Class_name
{

    // instance members
    Constructor name (parameter list)// as constructor…
    {
            //instance member initialization;
        Instance member= value/variable/expression;
    }
}
```

PARAMETERIZED CONSTRUCTOR- EXAMPLE:

```
class Abc
{
    int a;
    float b;
    String c;            // a & b are actual variable

    Abc (int m, float f)  // value of m & f is assign from main
    {
     a=m;                // m & f are formal variable
     b=f;
     System.out.println("a="+a+ "b="+b);
    }
}
```

```java
class Student
{
    int roll;
    String name;
    double per;

    Student(int r, String n, double p )
    {
        roll=r;
        name=n;
        per=p;
    }
        void display()
        {
            System.out.println("Roll no: "+roll);
            System.out.println("Name: "+name);
            System.out.println("per: "+per);
        }
}
class College
{
    public static void main(String arga[])
    {
        Student s=new Student(12,"KKWAGH",77.90);
        s.display();
    }
}
```

COPY CONSTRUCTOR- EXAMPLE:

```
class Abc
{
    int a,b,c;
                // a & b are actual variable

    Abc (Abc A1)     // A1 is formal object
    {
     c=A1.a+A1.b;
     System.out.print(c);

    }
}
```

COPY CONSTRUCTOR- EXAMPLE:

```
class Copy
{
        int x,y,z;

        Copy()
        {
        }
        Copy(Copy C)            // definition of copy constructor
        {
                z=C.x+C.y;
        }
        void display()
        {
                System.out.print("Addition: "+z);
        }
}
class MAIN
{
        public static void main(String str[])
        {
                Copy c1=new Copy();     // calling default constructor
                c1.x=10;
                c1.y=20;

                Copy c2=new Copy(c1);   // calling copy constructor
                c2.display();
        }
}
```

CONSTRUCTOR OVERLOADING

- The concept of overloading refers to the use of same name for different purposes. In simple language, **a class can have more than one constructor but with different parameters.**

- While having same name for all constructor, may perform different operation as defined by them.

- The constructor call depends on total parameters & its type within it.
  ie: con(int,int) & con(float,foat) are valid

CONSTRUCTOR OVERLOADING- EXAMPLE

```
class Abc
 {
      int n1,n2;
       float avg;
      Abc()
      {
       n1=5;
       n2=7;
      }
            Abc (int x)
            {
            n1=x;
            n2=x*5;
            }
            Abc(int x, int y)
            {
            n1=y;
            n2=x;
            }
}
```

```
class Box
{
    int height;
    int depth;
    int length;

    Box()
    {
       height = depth = length = 10;
    }
    Box(int x,int y)
    {
       height = x;
       depth = y;
    }
    Box(int x, int y, int z)
    {
       height = x;
       depth = y;
       length = z;
    }
}
class BoxClass
{
    public static void main(String args[])
    {
        Box a = new Box();            //statement1
        System.out.println("depth of a : "+a.depth);

        Box b = new Box(12,23);       //statement2
        System.out.println("depth of b : "+b.depth);

        Box c = new Box(99,84,36);    //statement3
        System.out.println("depth of c : "+c.depth);
    }
}
```

Program 2.5 Illustration of constructor overloading

```
import java.util.*;
class Area
{
        int len,bre,a;

        Area(int l,int b)        // constrcutor 1
        {
                len=l;
                bre=b;
        }
        int rect()               // normal method has return value
        {
                a=len*bre;
                return(a);       //System.out.println("Area of Rect="+a);
        }
        Area(int x)              // constructor 2
        {
                int t=x;
                t=t/2;
                System.out.println("Area of Tri= "+t);
        }
}
public class Rectangle
{
        public static void main(String add[])
        {
                Scanner s=new Scanner(System.in);

                System.out.println("Enter the len & bre of Rectangle");
                int l=s.nextInt();
                int b=s.nextInt();

                Area A1=new Area(l,b);  // 1st constructor called
                int R=A1.rect();
                System.out.println("Area of Rect="+R);

                Area A2=new Area(R);    // 2nd constructor called
        }
```

THIS

- Keyword in java.
- ‘This’ is reference variable that refers to the **current object of class.**
- ‘This’ keyword also refers current class instance variables.
- It is used to **avoid** the multiple variable declaration that hold same values.
- (specially about formal & instance variables)
- This will **reduce** the ambiguity between formal variable & instance variable of class.

```java
class Student
{
    int id;
    String name;
    Student(int id, String name)
    {
        id=id;  // will assign the NULL vaule
        name=name;
    }   // coz formal & local variable are same name
    void display()
    {
        System.out.println("id= "+id);
        System.out.println("name= "+name);
    }
}
class Isthis
{
    public static void main(String args[])
    {
    Student s=new Student(1103,"Angel");
    s.display();
    }
}
```

output- id=0 NULL name=0 NULL 1

```java
class Student
{
    int id;
    String name;
    Student(int id, String name)
    {
        this.id=id;
        this.name=name;
    }
    void display()
    {
        System.out.println("id= "+id);
        System.out.println("name= "+name);
    }
}
class Isthis
{
    public static void main(String args[])
    {
    Student s=new Student(1103,"Angel");
    s.display();
    }
}
```

output: id= 1103 name= Angel

THIS

- We can also refers **This()** as current class constructor.

- **this() constructor** will call the another parent constructor for reusing purpose

- **NOTE: this() can must (called) in another constructor before statements begin..**

- we can also used '**this(parameter)'** by calling constructor having single parameter.

Example……

EXAMPLE 1

```
import java.util.*;
class Area
{
        int side,volume;
        Area()
        {

                System.out.println("Formula: Volume of Cube= side*side*side");
        }
        Area(int  s)
        {
            this();                    // Area() not applicable here
                side=s;
                volume=s*s*s;
                System.out.println("Volume of Cube= "+volume);
        }

}
public class Overloading
{
        public static void main(String add[])
        {
                Area A=new Area(5);

        }
}
```

EXAMPLE 2

```
class Area
{
        int radius,height,base,rect;
        double circle;
        Area()
        {
                System.out.println("I am Default Constructor....");
        }
        Area(int r)
        {
            this();
                radius=r;
                circle=(3.14*r*r);
                System.out.println("Radius= "+radius+"\t\tArea of Circle= "+circle);
        }
        Area(int r,int H, int B)
        {
            this(r);
                height=H;
                base=B;
                rect=(height*base);
                System.out.println("Height= "+height+ " Base= "+base+"\tArea of Rectangle= "+rect);
        }
}
public class Overloading
{
        public static void main(String add[])
        {
                Area A=new Area(1,5,5);

        }
```

STRING & STRING BUFFER CLASS

STRING

- It is a Predefined Class describe in ‘java.lang’ package
- It is Sequence of characters (array of characters)
- First character present at 0 index.
- Value is final that can not be changed
- It is not a Datatype.
- Various operations are performed on it
- We pass array of String object as an Argument in Main() method

STRING CREATION

1) String (String value);

eg.: String s=new String("KKWPOLY");

2) String object_name = value;

eg.: String s="KKWP";

3) String (char chars[]);

eg.: char a[ ]={'a' , 'b' , 'c' , 'd'};

String s=new String (a);

4) String (char chars[], int startindex, int no_of_character);

eg: char v [ ]={'a' , 'b' , 'c' , 'd'};

String s=new String (v,2,2);

5) String(byte asciiChars[ ])

6) String(byte asciiChars[ ], int startIndex, int numChars)

```
class fun1
{
    public static void main(String obj[ ])
    {

        String str = new String ("Hello nashik");
        System.out.println(str);

        String s = "KKWAGH";
        System.out.println(s);

        char str1[ ]={'a','b'};
        String s1 = new String (str1);
        System.out.println(s1);


        char str2[] = {'a','b','c','d','e'};
        String s2 = new String (str2, 1, 3);
        System.out.println(s2);     // b c d

    }
}
```

Start index

no_of_char's

METHODS OF STRING CLASS

- **Name: length();**
- **Use: To find number of characters in string.**
- **Syntax: int length()**
- **Example : String s=“ KKWPOLY”;**

  int l=s. length();

METHODS OF STRING CLASS

- **Name: charAt();**
- **Use: To find character in string at particular index position**
- **Syntax : char charAt(int position)**
- **Example: String s="_K_K_W_POLY";**
  char ch=s. charAt (3);

METHODS OF STRING CLASS

- **Name: getChars()**

- **Use: To extract more than one character at a time, use the getChars( ) method**

- **Syntax: void getChars(int sourceStart, int sourceEnd, char target[], int targetStart)**

- **Example : String s = "This is a demo of the getChars method.";**

```
int start = 10;
int end = 14;
char buf[] = new char[end - start];
s.getChars(start, end, buf, 0);
```

METHODS OF STRING CLASS

- **Name: toCharArray()**

- **Use: To convert all the characters in a String object into a character array**

- **Syntax: char[ ] toCharArray( )**

- **Example : String s = “KKWP";**

```
char ch[]=s.toCharArray()
for(int i=0;i<ch.length;i++)
System.out.println(ch[i]);
```

METHODS OF STRING CLASS

- **Name: isEmpty()**
- **Use: To check string is <u>empty or not</u>**
- **Syntax: boolean isEmpty()**
- **Example: String s=“ ”;**

```
boolean b=s. isEmpty();
```

METHODS OF STRING CLASS

- **Name: equals()**

- **Use: To check two strings are equal or not (case sensitive)**

- **Syntax: boolean equals(Object obj)**

Eg: String s="KKWPOLY"; **String s1= "KKWPOLY";**

boolean b=s. equals(s1); **boolean b1=s1.equals("K.K.W.POLY");**

o/p : true **o/p : false**

METHODS OF STRING CLASS

- **Name: equalsIgnoreCase()**

- **Use: To check two strings are equal or not (case Insensitive)**

- **Syntax: boolean equalsIgnoreCase(Object obj)**

Eg: boolean b;

```
String s="KKWP";                    String s1= "KKwp";
b=s.equalsIgnoreCase(s1);    b=s1. equalsIgnoreCase("kKwP") ;
o/p : true                   o/p : true
```

METHODS OF STRING CLASS

- **Name: startsWith()**

- **Use: To check whatever string starts with particular sequence of character**

- **Syntax: boolean startsWith(String obj)**
  boolean startsWith(String obj, int index)

Eg: String s="KKWPOLY"; String s1= "KKWpoly";
b=s.startsWith("K"); b=s1.startsWith("W",2) ;
o/p: true o/p: true

METHODS OF STRING CLASS

- **Name: endsWith()**

- **Use: To check whatever string <u>ends</u> with particular sequence of character**

- **Syntax: boolean endsWith(String obj)**

Eg: String s="KKWPOLY";
b=s.endsWith("K") ;
o/p: false

String s1= "KKWpoly";
b=s1.endsWith("oly") ;
o/p: true

METHODS OF STRING CLASS

- **Name: indexOf()**

- **Use: To find <u>first index position</u> of particular sequence of character**

- **Syntax: int indexOf(String obj)**
  int indexOf(String obj,int index)

Eg: String s="KKWPOLY"; String s1= "KKWpolyKK";

Start index

pos= s.indexOf("KK"); pos=s1.indexOf("KK",4);

//1) 0
2) 7

METHODS OF STRING CLASS

- **Name: lastIndexOf()**

- **Use: To find <u>last index position</u> of particular sequence of character**

- **Syntax: int lastIndexOf(String obj)**
  int lastIndexOf(String obj,int index)

Eg: String s="KKWPOLY";
p=s.lastIndexOf("KW");

String s1= "KKWpolyKW";
p= s1.lastIndexOf("KW");

//1) 1
2) 7

METHODS OF STRING CLASS

- **Name: substring()**

s should be small //imp…

- **Use: To <u>find new string</u> from given index**

- **Syntax: String substring(int index)**

  String substring(int startindex, int endindex)

Eg: String s="KKWPOLY";

s2=s.substring(3);

:-1)=POLY

:-2)=KWPO

Start index

String s1= "KKWPOLY";

s2.substring(1,5) ; //5-1=4

Start index Last index

METHODS OF STRING CLASS

- **Name: concat()**
- **Use: To <u>join two string</u>**
- **Syntax: String concat(String obj)**

Eg: String s=“ KKWagh”; **String s1= “Polytechnic”;**

s2= s.concat(s1); **s2= s1.concat(“2.5”);**

o/p: KKWaghPolytechnic **o/p: Polytechnic2.5**

METHODS OF STRING CLASS

- **Name: compareTo()**
- **Use: To compare two strings as <u>less than, greater than or equal to.</u>**
- **Syntax: int compareTo(String obj)**

Eg: String s1="kkwp"; String s2= "kkwpoly";

int n= s1.compareTo(s2); n= s1.compareTo("MET");

-o/p: if s1 > s2, it returns positive number

if s1 < s2, it returns negative number

if s1 == s2, it returns 0

METHODS OF STRING CLASS

- **Name: toUpperCase()**

- **Use: To convert all characters in upper case**

- **Syntax: String toUpperCase()**

Eg: String s=“ KKWPOLYKK”; String s1= “KKWpoly”;

s2=s.toUpperCase(); s2=s1. toUpperCase();

METHODS OF STRING CLASS

- **Name: toLowerCase();**

- **Use: To convert all characters in Lower case**

- **Syntax: String toLowerCase()**

Eg: String s=“ KKWPOLYKK”; String s1= “KKWpoly”;

s2=s.toLowerCase(); s2=s1. toLowerCase();

METHODS OF STRING CLASS

- **Name: replace()**

- **Use: To <u>replace old sequence of character</u> with new sequence of character.**

- **Syntax: String replace(char oldch,char newch)**
  String replace((CharSequence oldText, CharSequence newText)

Eg: **String s="KKWPOLY";**
s1= s.replace("POLY","ENGG");

Old String New String

METHODS OF STRING CLASS

- **Name: trim()**
- **Use: To avoid leading and tailing space of String**
- **Syntax: String trim()**

Eg:

```
String s="      KKWPOLY      ";
s1=s.trim();
```

o/p: KKWPOLY

STRING BUFFER

- The StringBuffer class are used when there is a necessity to make a lot of modifications to Strings of characters.
- First character present at 0 index.
- Value is not final that can be changed.
- Present in java. lang package
- We cant use all methods of StringBuffer class in String class
- All methods of String class might be used in StringBuffer class (but not vise versa)

STRING BUFFER CREATION

Syntax…

```
StringBuffer(String value);
```

eg.

```
StringBuffer s = new StringBuffer ("WEL-COME");
// default Initial capacity = 16 characters
```

METHODS OF STRING BUFFER CLASS

- **Name: capacity()**
- **Use: To know the total allocated capacity(no. of char)**
- **Parameter: NA**
- **Return Type: int**

E.G **StringBuffer s= new StringBuffer ("KKWP");**

```
s.capacity();          //output:20
```

METHODS OF STRING BUFFER CLASS

- **Name: append()**

- **Use: To insert any value <u>at the end of string</u>**

- **Syntax: StringBuffer append(String str) // Object obj**

e.g. StringBuffer s= new StringBuffer ("KKWPOLY");

StringBuffer str= s.append("TECHNIC") ; or

s.append(123) ;

s.append(s1); // s1 is obj. of StringBuffer class

METHODS OF STRING BUFFER CLASS

- **Name: reverse();**

Syntax: StringBuffer reverse()

e.g. StringBuffer s= new StringBuffer ("KKWPOLY");
s.reverse() ;

o/p : YLOPWKK

METHODS OF STRING BUFFER CLASS

- **Name: insert();**

- **Use: To insert particular data value <u>at particular position</u>.**

Syntax : StringBuffer insert(int offset, String str)

E.G StringBuffer s= new StringBuffer ("KK POLY ");

s.insert(2, "WAGH") ;

Index position

o/p : KKWAGH POLY

METHODS OF STRING BUFFER CLASS

- **Name: setCharAt();**

- **Use: To <u>insert character</u> at particular position**

- **Syntax: StringBuffer setCharAt(int index, char ch)**

E.G. StringBuffer s= new StringBuffer ("KKWAG POLY");

s.setCharAt(5,'H') ;

Index position

o/p : KKWAGHPOLY

METHODS OF STRING BUFFER CLASS

- **Name: delete();**

- **Use: To <u>delete</u> sequence of character**

Syntax: StringBuffer delete(int startindex,int endindex)

StringBuffer s= new StringBuffer ("KKWAGHPOLY");

s.delete(2,5) ; note: 5-2=3

First index last index

o/p : KKHPOLY

METHODS OF STRING BUFFER CLASS

- **Name: deleteCharAt();**

- **Use: To delete character <u>at particular index</u> position**

- **Syntax: StringBuffer deleteCharAt(int index)**

Eg: StringBuffer s= new StringBuffer ("KKKWAGHPOLY");
s.deleteCharAt(2) ;

Index position

o/p : KKWAGHPOLY

ARRAY & VECTOR

ARRAY

- Collection of Homogeneous data elements, which residing into memory with linear fashion.
- Set Of Elements of similar data elements, having common variable name.
- e.g. integer, float, character, String
- First element of array present at 0th index location
- Array size indicate total no. of allowable values[size].
- Length variable is used to know total element of array
- Types: 1D and 2D

1D ARRAY CREATION

Syntax:

```
datatype arrayname[ ] = new datatype[int size];
```

OR

```
datatype [ ]arrayname = new datatype[int size];
```

Size: Maximum elements stored in array

e.g. `int a[ ]= new int[5];` Or `int a[];`

`a= new int[5];`

`char b[ ]= new char[3];`

`float c[ ]= new float[10];`

ARRAY INITIALIZATION

Syntax:

Compile Time:

```
datatype arrayname[] = { list of elements };
```

Run time:

```
datatype arrayname[]=new datatype[size];
```

```
e.g. int a[]= {10,20,5,20};
     char b[]= { 'a' , 'b' , 'c' , 'd'};

int a[4]= {10,20,50,20};
```

Will give error at run time

ACCESSING ARRAY ELE.

```
public static void main(String
    args[])
{
    int a[]={10,20,30,40,50};

    System.out.println(a[0]);
    System.out.println(a[2]);
    System.out.println(a[4]);
}
```

```
public static void main(String args[])
{
    int a[]={10,20,30,40,50};
    int len=a.length;
    for (int i=0; i<len; i++)
    {       System.out.println(a[i]);
    }
}
```

READING ARRAY ELE.

```
public static void main(String args[])
{
    int arr[]=new int[5];                    // declaration
    Scanner s=new Scanner(System.in);
    System.out.println("Enter the 5 element of array..");
    for(int i=0; i<arr.length; i++)
    {
     arr[i]=s.nextInt();
    }
}
```

```
//Find maximum number from array
  class FindMax
  {
      public static void main(String args[])
      {
          int x[] = {8,4,6,2,0,3,11,5,1}; //array defined
          int max = x[0];  //consider first no. as max

          System.out.print("Array is : ");
          for(int i=0;i<x.length;i++)  //print array
              System.out.print(x[i]+", ");

          for(int i=1;i<x.length;i++)
          {
              if(x[i] > max) //check each no. with max
                 max = x[i];
          }
          System.out.println("\nMaximum is "+max);
      }
  }
```

Program 3.1 Finding maximum number from the array

Output:

```
Array is : 8, 4, 6, 2, 0, 3, 11, 5, 1,
Maximum is 11
```

ARRAY

HomeWork:

1) Write a JAVA code to accept 5 random element of array from user and sort them into Ascending order

2) Write a JAVA code to accept 5 random element of array from user and Search any specific element of that array with his location.

3) Write a JAVA code to accept and perform SUM of ARRAY of 5 element.

2D ARRAY

Syntax:

datatype arrayname[ ][ ]= new datatype [int row][int col];

e.g. int a[ ][ ]= new int[3][2]; //declaration…

Column's

Row's

0,0	0,1
1,0	1,1
2,0	2,1

2D ARRAY INITIALIZATION

Syntax:

Compile Time:

```
Datatype arrayname[][] = { list of elements};
```

Run Time:

```
Datatype arrayname[][]=new datatype[row][col];
```

e.g.
```
int a[][]= { {5,7},  {8,3},  {1,2} };
```

OR

```
int a[][]={   {5,7},
              {8,3},
              {1,2}   };
```

5	7
8	3
1	2

2ARRAY EXAMPLE

Assignment--

1) **WAP to perform <u>addition of two 2x2 matrix</u>, and display the addition**

2) **WAP to accept & compare Two 3x3 matrix.**

3) **WAP to inverse the 3x3 matrix of given inputted matrix**

Note: to access 2D Array, we probably used <u>2 for loop</u> for accessing rows & columns of array….

SORTING & SEARCHING OF ARRAY

-Java provide ready made method to sort, equal and search the array elements

-Arrays class used under .util package

-the method sort(); equals(); & binarySearch(); are used with class Arrays….

Syntax..

No. to be searched

Sorting

```
int a[] = {5,9,6,4,7};
Arrays.sort(a);
```

Searching

```
int arr[] = {0,2,3,5,8,9};
int val = Arrays.binarySearch(arr, 8);
```

Equality

```
int x[] = {8,4,6,2,5};
int y[] = {8,4,6,2,5};
if(Arrays.equals(x,y))    //statement1
   System.out.println("Arrays are equal");
```

```java
import java.util.*;
class sorting
{
    public static void main(String args[])
    {
        int a[]={55,44,77,22,11};
        int len=a.length;
        System.out.println("given array...");
        for(int i=0; i<len; i++)
        {
            System.out.println(a[i]);
        }
            System.out.println("searching....");
            int val=Arrays.binarySearch(a,77);
            System.out.println("Element found " +val);

        System.out.println("After sorting array became...");
        Arrays.sort(a);
        for(int i=0; i<len; i++)
        {
            System.out.println(a[i]);
        }
    }
}
```

ARRAY OF OBJECT

Syntax:

ClassName Object[]= new className [int size];

- **To access methods and members of class, it must initialize all members using constructor .**

e.g. **Abc obj[]= new Abc[5];**

```
for (int i=0; i<=5; i++)
{
    obj[i]=new Abc();
            // constructor will call 5 times
}
```

VECTOR

- It is a class in java Available in java.util package
- Creating dynamic array known as 'Vector' which can hold the element of *any type and any number*
- Elements may be homogeneous or heterogeneous
- Vector index start with 0th
- Vector used to utilize the memory allocation.
- It create memory space at RUN TIME

VECTOR : CONSTRUCTOR

1) Vector() //default capacity is 10

Eg: Vector V=new Vector();

2) Vector (int capacity) //declaring vector with manual capacity

Eg: Vector V=new Vector(5);

3) Vector (int capacity, int incre)

Eg: Vector V=new Vector(5, 2);

// if size > default_capacity (5) then, capacity incremented by 2

METHODS OF VECTOR CLASS

- **Name: capacity();**
- **Use: To check total default capacity of vector**
- **Syntax: int capacity()**

E.G Vector v=new Vector(); Vector v=new Vector(5);

v.capacity(); **v.capacity();**

// output is 10 // output is 5

METHODS OF VECTOR CLASS

- **Name: addElement();**

- **Use: to <u>add element to the last of the vector</u>**

- **void addElement(Object obj)**

E.G: **Vector v=new Vector();**

```
v.addElement(12);
v.addElement("Hello");
v.addElement("3.14");
```

METHODS OF VECTOR CLASS

- **Name: size();**
- **Use: To check current total number of elements present in vector**
- **Syntax : int size()**

E.G **Vector v=new Vector();**

int l=v.size();

METHODS OF VECTOR CLASS

- **Name: trimToSize();**
- **Use: It set vector capacity equal to current size of elements**
- **Parameter: --**
- **Return Type: void**

E.G **Vector v=new Vector();**

v.trimToSize();

METHODS OF VECTOR CLASS

- **Name: isEmpty()**
- **Use: To check vector is <u>empty or not</u>**
- **Syntax : boolen isEmpty()**

E.G **Vector v=new Vector();**

boolean b=v.isEmpty();

METHODS OF VECTOR CLASS

- **Name: contains()**

- **Use: To check whether element contained (present) in vector or not.**

- **Syntax: Boolean contains(Object obj)**

E.G

```
Vector v=new Vector();
v.addElement("hello");
boolean b= v.contains("hello");
```

METHODS OF VECTOR CLASS

- **Name: elementAt();**

- **Use: To <u>find</u> element in vector at particular <u>index position</u>**

- **Syntax: Object elementAt(int index)**

E.G

```
Vector v=new Vector();
Object o= v.elementAt(2)
System.out.print(o);        // give the value of 2nd index
```

METHODS OF VECTOR CLASS

- **Name: insertElementAt()**
- **Use: To insert element at specific index position of vector**
- **Syntax: void insertElementAt(Object obj, int index)**

E.G **Vector v=new Vector();**

v.insertElementAt("Hello", 2);

METHODS OF VECTOR CLASS

- **Name: removeElementAt()**
- **Use: To Delete element of vector from particular index**
- **Syntax: void removeElementAt(int index)**

E.G **Vector v=new Vector();**

v.removeElementAt(2);

METHODS OF VECTOR CLASS

- **Name: removeAllElements(**
- **Use: To Delete all elements of vector**
- **Syntax: void removeAllElements()**

E.G **Vector v=new Vector();**

v.removeAllElements();

METHODS OF VECTOR CLASS

- **Name: firstElement()**
- **Use: To find first element of vector**
- **Syntax: Object firstElement();**

E.G **Vector v=new Vector();**

Object o = v.firstElement();

METHODS OF VECTOR CLASS

- **Name: lastElement()**
- **Use: To find Last element of vector**
- **Syntax: Object lasttElement();**

E.G **Vector v=new Vector();**

Object o = v.lastElement();

METHODS OF VECTOR CLASS

- **Name: indexOf()**
- **Use: To find index position of particular element**
- **Syntax: int indexOf(Object o)**

E.G **Vector v=new Vector();**

```
int a = v.indexOf("Hello");
```

METHODS OF VECTOR CLASS

- **Copy one Vector Element into another Vector….**

Syntax…

```
Vector v1=new Vector();
Vector v2=new Vector();
    v1.addElement("One");
    v1.addElement(2);
    v1.addElement(3.00);
        v2=v1;
            System.out.println(v2);
```

SAMPLE CODE FOR VECTOR

```
import java.util.*;
class vec
{
  public static void main(String args[])
  {
        Vector v=new Vector();

        System.out.println("size :"+v.size());
        System.out.println("capacity :"+v.capacity());

        v.addElement("Hello");
        v.addElement(12);
        v.addElement("tush28");
        v.addElement('A');
                                  //printing vector elements
        System.out.println("elements are: "+v);

        System.out.println("size :"+v.size());
        System.out.println("capacity :"+v.capacity());

        v.insertElementAt("kkwagh",2);
        v.removeElementAt(0);

                                  //printing vector elements
        System.out.println("elements are: "+v);

        System.out.println("Index of= "+v.indexOf("tush28"));

  }
}
```

COMMAND LINE ARGUMENT

- User can pass input parameter while executing program using command line

- **Input value are stored in main method** parameter in the form of string.

- First value store in 0th index location

- E.g how to Execute… in command prompt?

- javac <file_name.java>
- java <file_name> list of input parameter
- Eg: java command hello friends how are you

```
// Using command line arguments
  class CommandLine

  {
       public static void main(String args[])
       {
            System.out.println("Arguments are : ");
            for(int i=0;i<args.length;i++)
            {
               System.out.print("args["+i+"] = ");
               System.out.println(args[i]);
            }
       }
  }
```

Program 3.12 Demonstration of command line arguments

This program will be compiled and they executed as

```
java CommandLine hi This is Java 6
```

We will get the output:

```
Arguments are :
args[0] = hi
args[1] = This
args[2] = is
args[3] = Java
args[4] = 6
```

Command Line Execution- Addition of two no…

```
class command
{
   public static void main(String arg[])
   {
        int a=Integer.parseInt(arg[0]);
        int b=Integer.parseInt(arg[1]);
        System.out.println("Addition= "+(a+b));
   }
}
```

Garbage Collection

- The ‘new’ operator is used to create memory space for Object of class.
- But like C++ java doesn’t support ‘delete’ operator or any destructor() for releasing the space acquire by Object.
- When we finishing object reference, it simply stop referring.
- But memory retain used by object.
- Object were no longer used are referred as ‘Garbage’.
- Garbage collection is the process of clean up (Freeing) memory by deallocation object that is no longer exist.

Garbage Collection

finalize() method

- The Java's automatic garbage collector calls the finalize( ) method for destroying the object.

- The finalize() method is defined in the Object class

```
protected void finalize( ) throws Throwable
{
      // finalization code
}
```

WRAPPER CLASS

- **Wrapper class** is defined as a class in which a **primitive data type** is wrapped up.
- These **primitive wrapper classes** are used to represent primitive data type values **as objects**.
- **Used for 'pass by reference'**
- Eg: Integer wrapper class hold the value of int as an object
- **Each wrapper class has different constants and methods.**
- Total 8 Wrapper class Available in Java
- Available in java.lang package

WRAPPER CLASSES IN JAVA

Data Type	Wrapper Class
byte →	Byte
short →	Short
int →	Integer
long →	Long
float →	Float
double →	Double
char →	Character
boolean →	Boolean

WRAPPER CLASS: INTEGER FAMILY

Constructors

1) **Integer (int value)**

   1) Integer io=new Integer (11);

OR

2) **Integer (String str) throws NumberFormatException**

   2) Interger io=new Integer ("11");

….respectively, For Byte, Short & Long

- **Methods**

  Value can be convert to any data type using "###Value()" method except boolean and character where ### is name of data type.

  e.g. byteValue(), intValue(), longValue() etc…

Wrapper Class: Example

```
int i = 26;     // Primitive data type 'int'
//  Integer Wrapper class instantiation
Integer i_Obj = new Integer(i);
// Unwrapping primitive data 'int' from wrapper object
int i2 = i_Obj.intValue();
```

Wrapper Class: Integer Family Methods

The Java Wrapper Classes include various useful methods:

- Convert string value into integer : static int parseInt(String str)

  `e.g. int value = Integer.parseInt("33");` //sets value to 33.

- Converting a value to a String: static toString(int value)

  `e.g. String str = Integer.toString(33);` //sets str to the String "33".

- Getting a hexadecimal value of int value:
  - static String toHexString(int b)
- Getting a octal value of int value:
  - Static String toOctalString(int b)
- Getting a binary value of int value:
  - Static String toBinaryString(int b)

  `e.g. String str = Integer.toBinaryString(12);` // creates a binary of 12 is 1100

Wrapper Class: Integer Family Methods

- To compare two objects values :
  - int compareTo(Integer obj)

    e.g. int value = intobj1.compareTo(intobj2);

    Will return 0= both equal, -ve= intobj1 less, +ve= intobj1 greater

- To check the equality of two objects values:
  - boolean equals(Integer obj)

    e.g. boolean v = intobj1.equals(intobj2); // Will return true= equal or false=not equals

- To get Integer object that has been initialised with the supplied value.
  - static Integer valueOf(int b)

    e.g. Integer obj=Integer.valueOf(2);

WRAPPER CLASS: FLOAT FAMILY

Constructors

1) **Float(Float value)**
   1) **Float fo=new Float (11.43f);**
2) **Float (String str)**
   2) **Float fo=new Float ("11.43");**
3) **Float(double value)**
   3) **Float fo=new Float (11.43);**

…..respectively for Double

- **Methods**

Value can be convert to any data type using "###Value()" method except boolean and character where ### is name of data type.

E.G floatValue(), doubleValue() etc..

WRAPPER CLASS: FLOAT FAMILY

- Convert string value into float : static float parseFloat(String str)
  `e.g. float v = Float.parseFloat(11.33f);`

- Converting a value to a String: static toString(float value)
  `e.g. String str = Float.toString(11.33f);`

- To compare two objects values :
  - int compareTo(Integer obj)
    `e.g. int value = intobj1.compareTo(intobj2);`
    Will return 0= both equal, -ve= intobj1 less, +ve= intobj1 greater

- To check the equality of two objects values:
  - boolean equals(Integer obj)

  `e.g. boolean v = intobj1.equals(intobj2);`

  // Will return true= equal or false=not equals

WRAPPER CLASS: BOOLEAN

Constructor

1) **Boolean (boolean value)**

Eg: 1) Boolean B=new Boolean(true);

- **Methods**

boolean equals(boolean value) ** true or false

static String parseBoolean(String value)

```
eg:    Boolean b1=new Boolean(false);
  Boolean b2=new Boolean(true);
  boolean result=b1.equals(b2);            //  return true or false
  boolean b3=Boolean.parseBoolean("true");     // return string true
```

WRAPPER CLASS: CHARACTER

Constructor

1) **Character (char value)**

   eg: **Character C=new Character ('A');**

WRAPPER CLASS: CHARACTER

Name	Return Type
isLetter(char ch)	boolean
isDigit (char ch)	
isWhitespace(char ch)	
isUpperCase(char ch)	
isLowerCase(char ch)	
toUpperCase(char ch)	char
toLowerCase(char ch)	
toString(char ch)	String

Note : All methods are static

Wrapper Class: Character

Sample code…

```
class wrap1
{
  public static void main(String args[])
  {
    Character in=new Character('a');  // declaration of wrapper class

    int max=in.MAX_VALUE;

    System.out.println("MAX "+max);

    System.out.println("Letter " +Character.isLetter(in));

  }
}
```

END OF CHAP 2

Thank You...